

CSU34041

Transactions and Concurrency Control

Yvette Graham
ygraham@tcd.ie



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin

Overview

Transaction

- Properties / Failure

Concurrency Control

- Lost Updates
- Temporary Updates
- Incorrect Summary

Schedules

- Schedule Conflicts
- Serial Schedules
- Issues with Serial Schedules

Serializability

Result Equivalence

Conflict Equivalence

Locking

Binary Lock

Read/Write Lock

Lock Conversion

Two Phase Locking

- Problems/Limitations

Deadlock

Deadlock Prevention Protocols

- No Waiting
- Wait-die
- Wound-wait
- Cautious Waiting

Deadlock Detection

Starvation

Starvation Solutions

Timestamp Ordering

Recommended reading: Chapters 20 & 21, Fundamentals of DB Systems Elmasri & Nevanthe 7th edition



Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

Introduction



Introduction

- Airline reservations, Credit Card Processing, Online Shopping etc. are examples of very large **real-world database systems**
 - can potentially have thousands of users performing operations at the same time
 - operations can be complex, involving a number of separate actions
- Create booking, update remaining seats, charge credit card, issue confirmation....



Introduction

Transactions and **Concurrency Control** are ways in which a DB manages complex processes and multi-user access

- Transaction
 - a logical unit of DB processing that is completed in its entirety to ensure correctness
- Concurrency Control
 - used when two operations try to access the same data at the same time



Transactions



Transactions

- A transaction includes one or more DB access operations
 - Insertion, Deletion, Modification, Retrieval
- Transactions where the DB operations retrieve data, but don't update any information
 - *Read-Only* Transactions
- Transactions which update the DB
 - *Read-Write* Transactions



Transactions

- Transactions take the form:

START_TRANSACTION

 READ or WRITE operation 1

 READ or WRITE operation 2

 READ or WRITE operation n

END_TRANSACTION

COMMIT or ROLLBACK



Transactions

- The end of a transaction is signalled by either
 - commit (successful termination)
 - rollback or abort (unsuccessful termination)
- **Commit** completes the current transaction, making its changes permanent
- **Rollback** will undo the operations in the current transaction, cancelling the changes made to the DB



Transaction Failure

- There are a number of reasons why a transaction might fail
 - System **crash**
 - Hardware, Software or Network Error
 - Transaction **error**
 - i.e. divide by zero, incorrect attribute reference, incorrect parameter value
 - **Checks** failing within the transaction
 - i.e. insufficient funds to make a withdrawal



Transaction Properties



Transaction Properties

All transactions should possess the **ACID** properties

- **A**tomicity
- **C**onsistency preservation
- **I**solation
- **D**urability (or permanency)



Transaction Properties

Atomicity

- A transaction is an atomic unit of processing
- It should either be performed in its entirety or not performed at all



Transaction Properties

Consistency preservation

- A transaction should preserve the consistency of the DB
- It should take the database from one consistent state to another



Transaction Properties

Isolation

- A transaction should appear as though it is being executed in isolation
- The execution of a transaction should not be interfered with by any other transactions executing concurrently



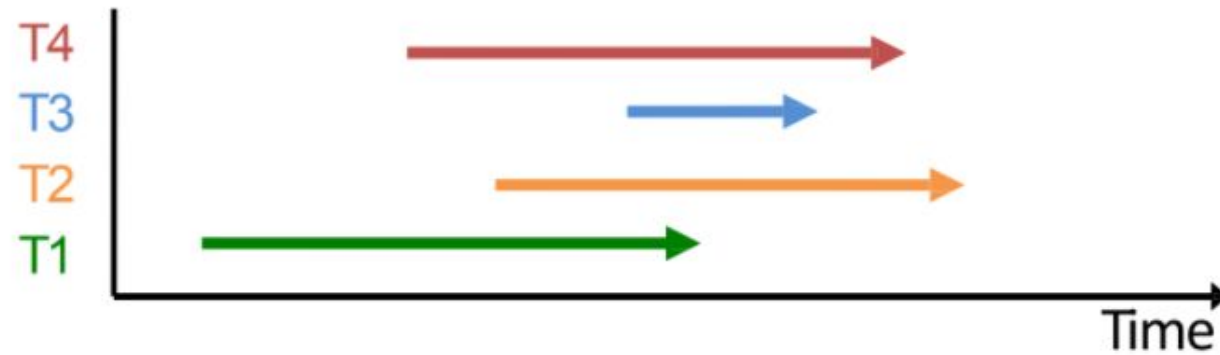
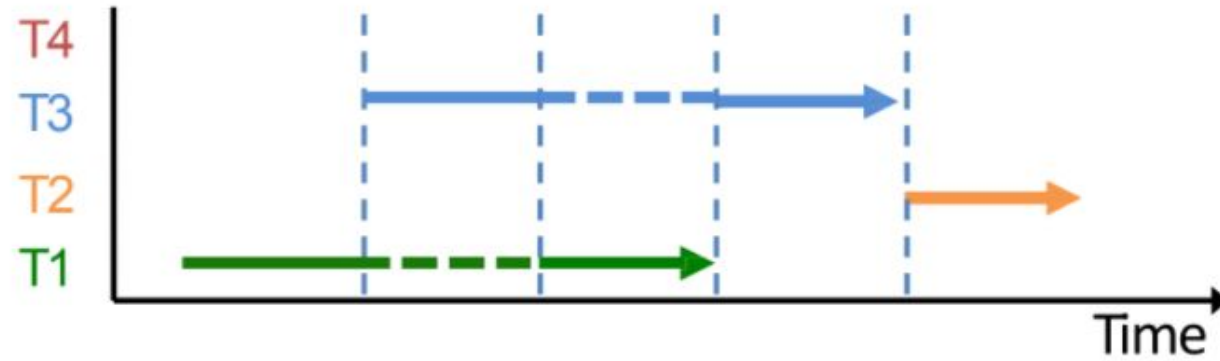
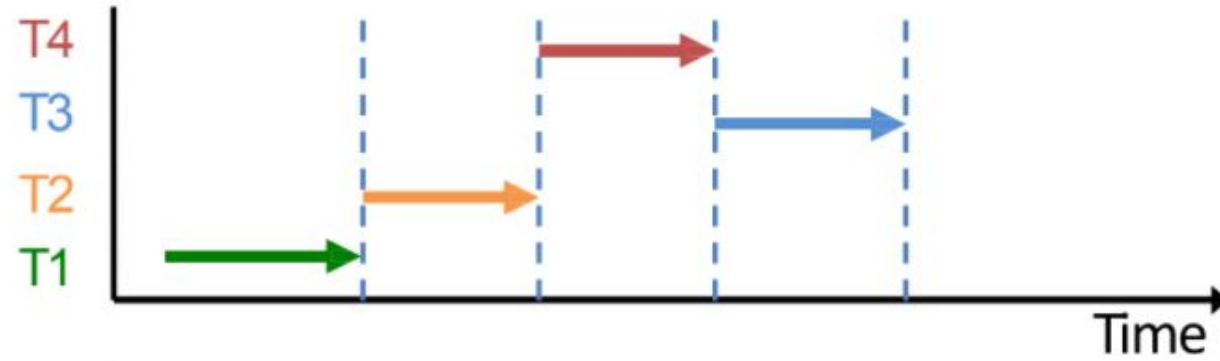
Transaction Properties

Durability (or permanency)

- The changes applied by a committed transaction must persist in the database
- These changes must not be lost because of any failure



Example



Concurrency Control



Concurrency Control

- Consider the following two transactions:

Flight Transfer

```
read(reserved_seats_X)
reserved_seats_X = reserved_seats_X - n
write(reserved_seats_X)
read(reserved_seats_Y)
reserved_seats_Y = reserved_seats_Y + n
write(reserved_seats_Y)
```

Flight Reservation

```
read(reserved_seats_X)
reserved_seats_X = reserved_seats_X + m
write(reserved_seats_X)
```

- A number of problems can occur if these two simple transactions are run concurrently



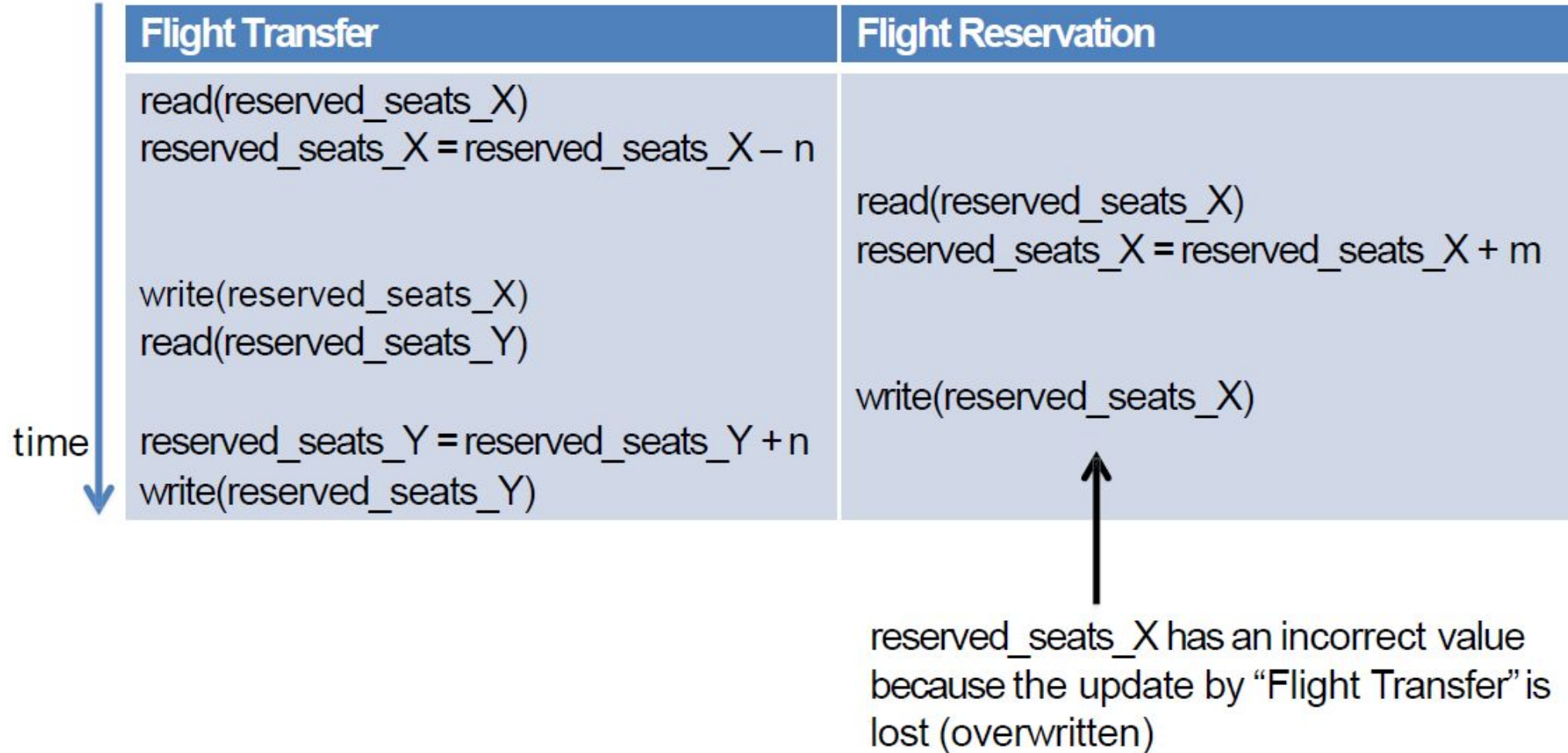
Lost Update

Lost update can occur when two or more transactions:

- access the same data item
- are executed concurrently
- are interleaved in such a way that results in an incorrect value being written to the database



Lost Update

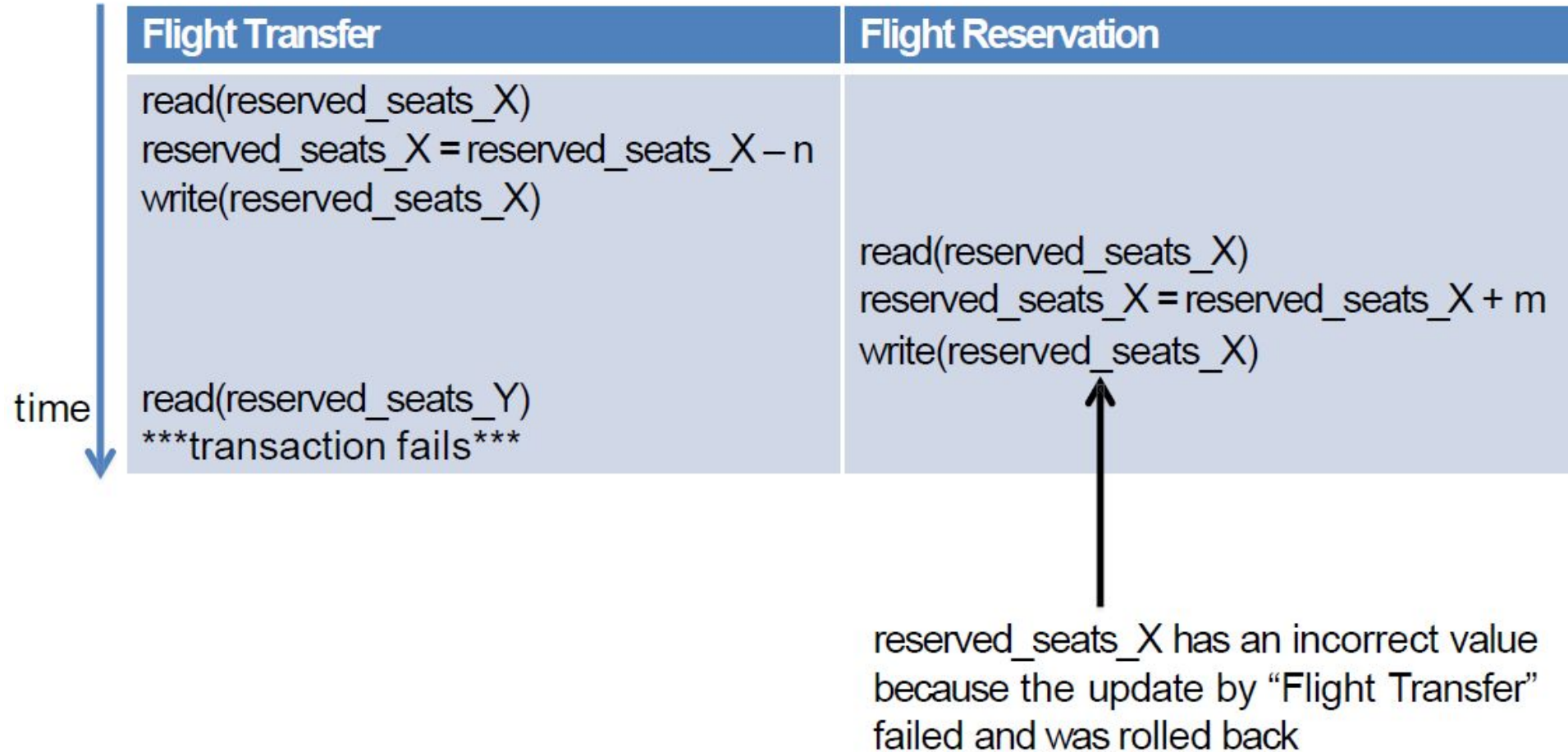


Temporary Update

- This occurs when two or more transactions:
 - access the same data item
 - are executed concurrently
 - are interleaved
 - one transaction fails and must Rollback
- This is also known as the “Dirty Read”



Temporary Update



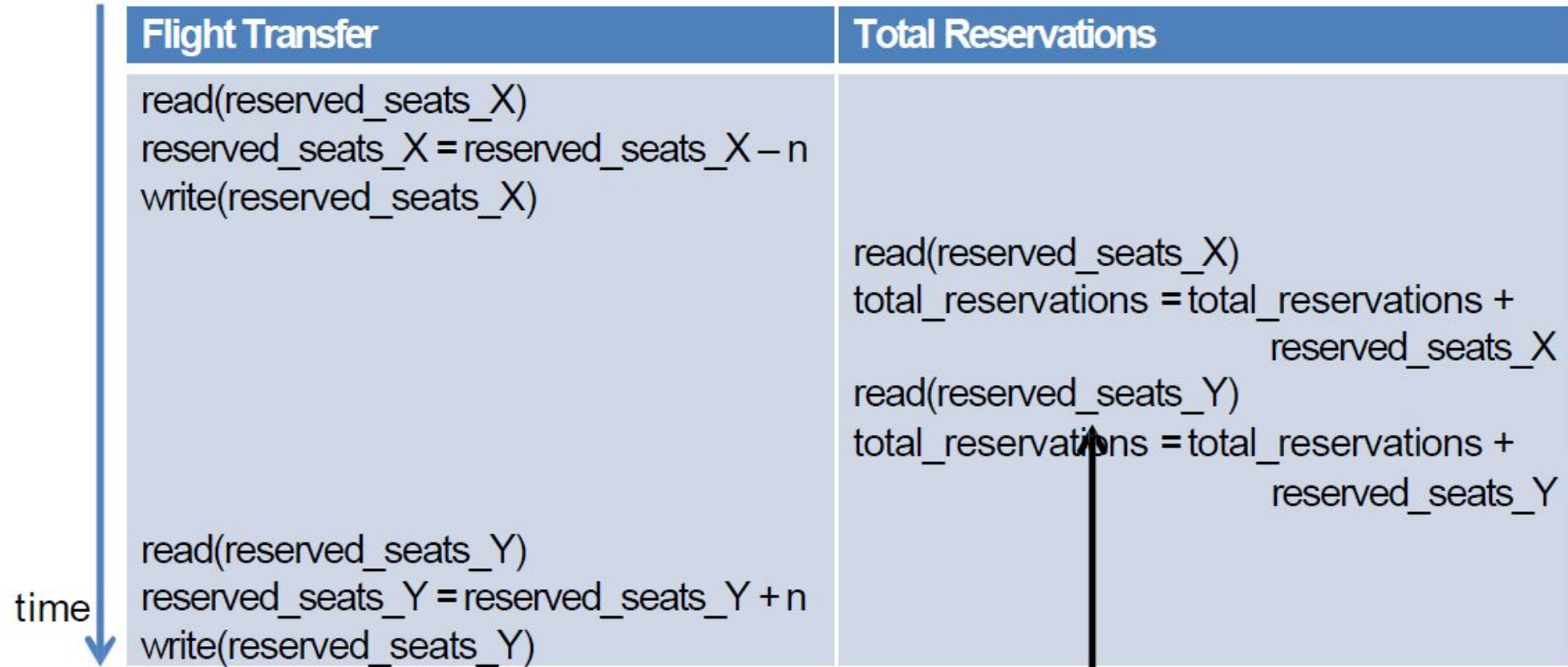
Incorrect Summary

This occurs when two or more transactions:

- access the same data item
- are executed concurrently
- are interleaved
- one transaction is calculating an aggregate summary on attributes while another transaction is updating those attributes



Incorrect Summary



reserved_seats_Y has an incorrect value because the update by "Flight Transfer" hasn't happened yet

Schedules



Schedules

- When transactions are executing concurrently in an interleaved fashion, the order of execution of operations is called the *schedule*
 - Schedule S of n transactions $T_1, T_2, \dots, T_n$ is the ordering of the operations within those transactions
- Operations from different transactions can be interleaved
 - The operations from each transaction T_i must appear in the same order in S , that they do in T_i



Schedules

- A schedule S is generally written:
 - $S_a: O_i(X); O_i(Y); O_j(X); \dots ; O_n(Y);$
 - where $O_i(X)$ indicates either a read or write operation executed by a transaction T_i on a data item X
 - and $O_n(Y)$ indicates either a read or write operation executed by a transaction T_n on a data item Y



Schedules

Flight Transfer (Transaction 1)	Flight Reservation (Transaction 2)
time ↓ <pre>read(reserved_seats_X) reserved_seats_X = reserved_seats_X - n write(reserved_seats_X) read(reserved_seats_Y) reserved_seats_Y = reserved_seats_Y + n write(reserved_seats_Y)</pre>	<pre>read(reserved_seats_X) reserved_seats_X = reserved_seats_X + m write(reserved_seats_X)</pre>

Is denoted as?

- A schedule S is generally written:
 - $S_a: O_i(X); O_i(Y); O_j(X); \dots; O_n(Y);$
 - where $O_i(X)$ indicates either a read or write operation executed by a transaction T_i on a data item X
 - and $O_n(Y)$ indicates either a read or write operation executed by a transaction T_n on a data item Y



Schedules

Flight Transfer (Transaction 1)	Flight Reservation (Transaction 2)
read(reserved_seats_X) reserved_seats_X = reserved_seats_X - n write(reserved_seats_X)	
read(reserved_seats_Y) reserved_seats_Y = reserved_seats_Y + n write(reserved_seats_Y)	read(reserved_seats_X) reserved_seats_X = reserved_seats_X + m write(reserved_seats_X)

time ↓

Is denoted as?

$S_1: R_1(\text{reserved_seats_X}); W_1(\text{reserved_seats_X});$
 $R_2(\text{reserved_seats_X}); W_2(\text{reserved_seats_X});$
 $R_1(\text{reserved_seats_Y}); W_1(\text{reserved_seats_Y});$

- A schedule S is generally written:
 - $S_a: O_i(X); O_i(Y); O_j(X); \dots; O_n(Y);$
 - where $O_i(X)$ indicates either a read or write operation executed by a transaction T_i on a data item X
 - and $O_n(Y)$ indicates either a read or write operation executed by a transaction T_n on a data item Y



Schedule Conflict

- Two operations in a schedule are said to **conflict** if:
 - 1) they belong to different transactions
 - 2) they access the same item X
 - 3) and at least one of the operations is a write(X)
- Intuitively, two operations are conflicting if **changing their order can result in a different outcome**
 - or cause one of the concurrency issues we have already discussed

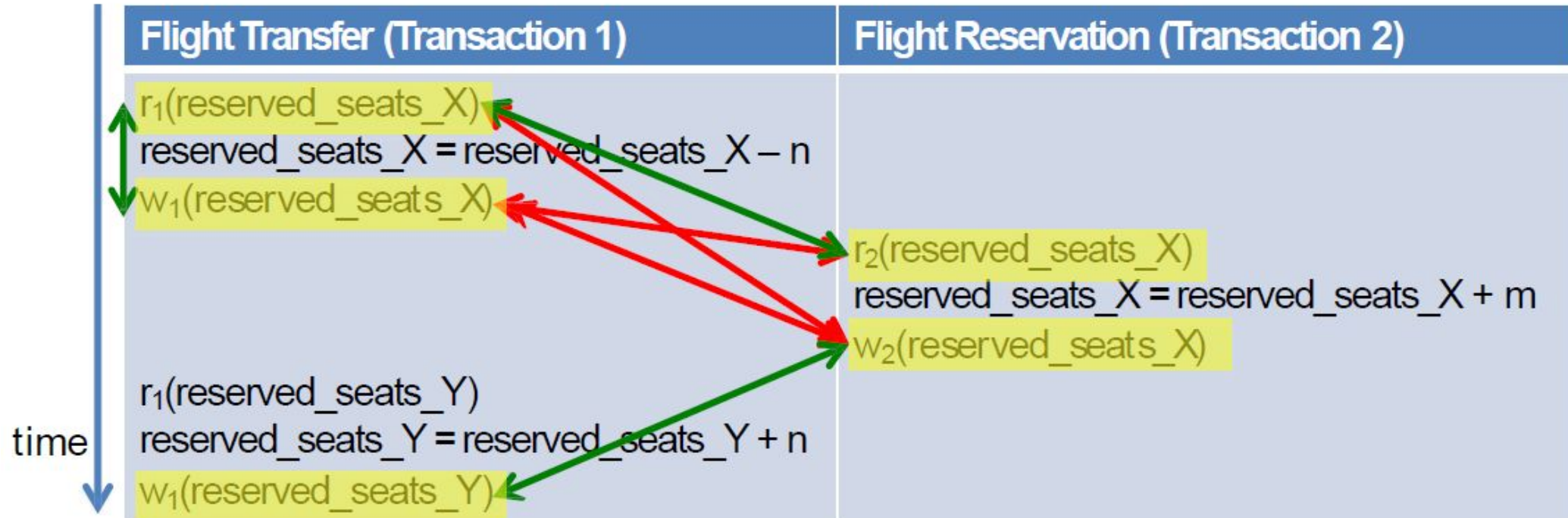


Schedule Conflict

- Operations O_i and O_n of transactions T_i and T_n respectively on data item Q .
 1. $O_i = \text{read}(Q)$, $O_n = \text{read}(Q)$ O_i and O_n don't conflict.
 2. $O_i = \text{read}(Q)$, $O_n = \text{write}(Q)$ They conflict.
 3. $O_i = \text{write}(Q)$, $O_n = \text{read}(Q)$ They conflict
 4. $O_i = \text{write}(Q)$, $O_n = \text{write}(Q)$ They conflict



Schedule Conflict



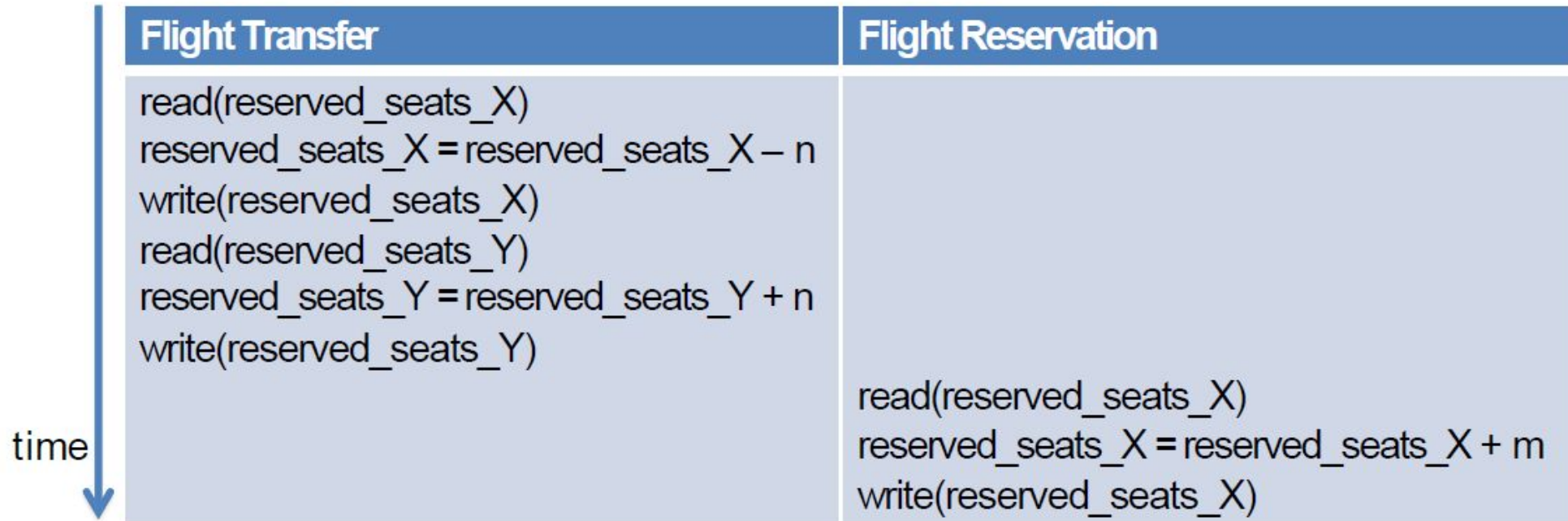
$S_1: R_1(\text{reserved_seats_X}); W_1(\text{reserved_seats_X});$
 $R_2(\text{reserved_seats_X}); W_2(\text{reserved_seats_X});$
 $R_1(\text{reserved_seats_Y}); W_1(\text{reserved_seats_Y});$

Serial Schedules



Serial Schedules

- In a *serial schedule* the operations of each transaction are executed consecutively, without any interleaved operations



Serial Schedules

- Formally, a schedule S is serial if, for every transaction, T , participating in the schedule, all operations of T are executed **consecutively**
 - Otherwise the schedule is **nonserial**
- In a serial schedule, only one transaction is executed at a time
 - the commit (or abort) of the active transaction initiates execution of the next transaction



Serial Schedules

- An assumption that can be made is: every serial schedule is correct
 - All transactions should be independent (Isolation)
 - Each transaction is assumed to be corrected if executed on its own (Consistency preservation)
 - Hence, the ordering of transactions in a serial schedule does not matter
 - Once every transaction is executed from beginning to end

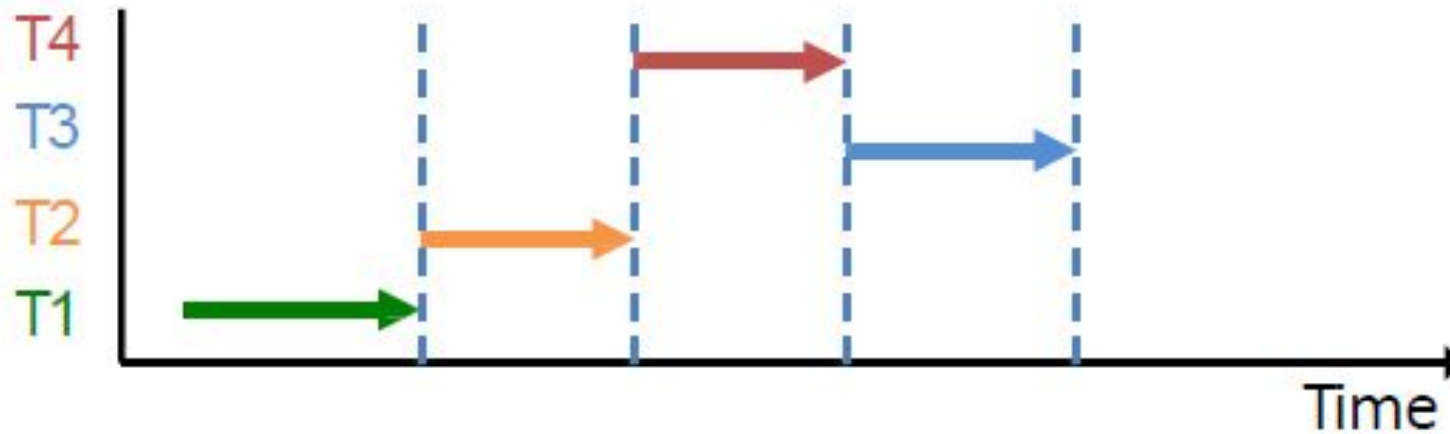
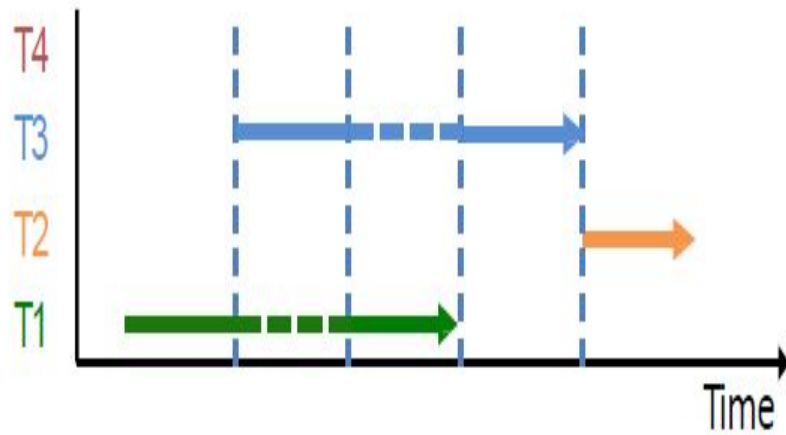


Issues with Serial Schedules

- That is great ... but
 - Serial schedules limit concurrency
 - If a transaction is waiting for an operation to complete, it is not possible to switch processing to another transaction
 - If a transaction is very long, all other transactions must wait
- Hence, serial schedules are considered unacceptable in practice



Example



Serializability & Result Equivalence

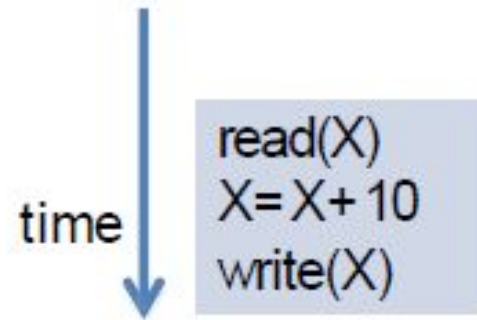


Serializability

- However, it can be determined which other schedules are *equivalent* to a serial schedule, those schedules can be allowed to occur
- How is equivalence measured?
 - Result equivalence
 - Conflict equivalence
- If a non serial schedule meets these criteria, it is said to be serializable



Result Equivalence



- Two schedules are said to be *result equivalent* if they produce the same final state of the DB
 - This is the simplest notion of equivalence



Result Equivalence



- However two schedules could coincidentally produce the same result

Conflict Equivalence

- Two schedules are said to be conflict equivalent if the order of any two conflicting operations is the same in both schedules
- Reminder: two operations in a schedule are said to conflict if:
 - They belong to different transactions AND
 - They access the same item X AND
 - At least one of the operations is a write(X)



Conflict Equivalence

- If two conflicting operations are applied in different orders in two schedules, the effect can be different on the DB
 - Hence, the schedules are not conflict equivalent
- Example: a read operation $r1(X)$ and a write operation $w2(X)$ in two schedules S_A and S_B
 - $S_A: r1(X), w2(X)$
 - $S_B: w2(X), r1(X)$
 - The value read by $r1(X)$ may be different in the two schedules as it may have been updated by the write
 - S_A and S_B are not conflict equivalent

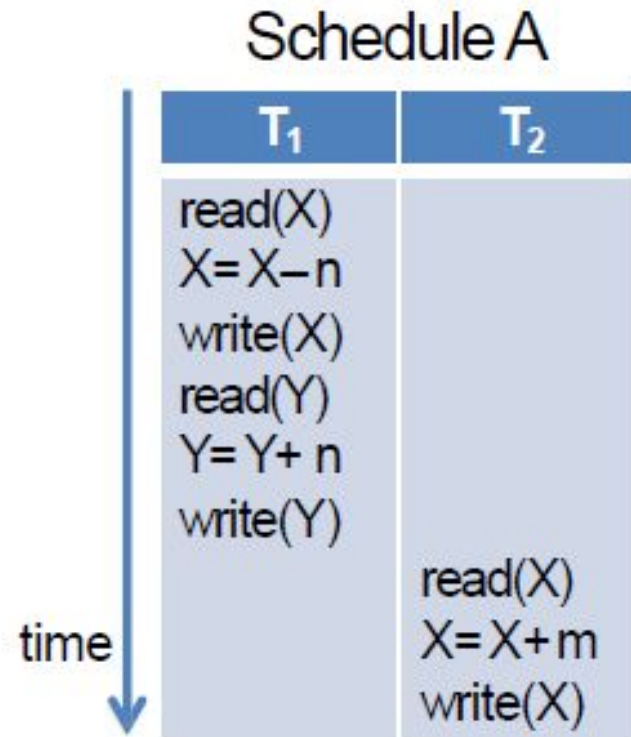


Conflict Equivalence

- Similarly, if two write operations occur in the order:
 - $S_A: w1(X), w2(X)$
 - $S_B: w2(X), w1(X)$
 - The next $r(X)$ operation in the two schedules will read potentially different values
 - If these are the last two operations in the schedules, the final value of item X in the DB will be different



Example 1



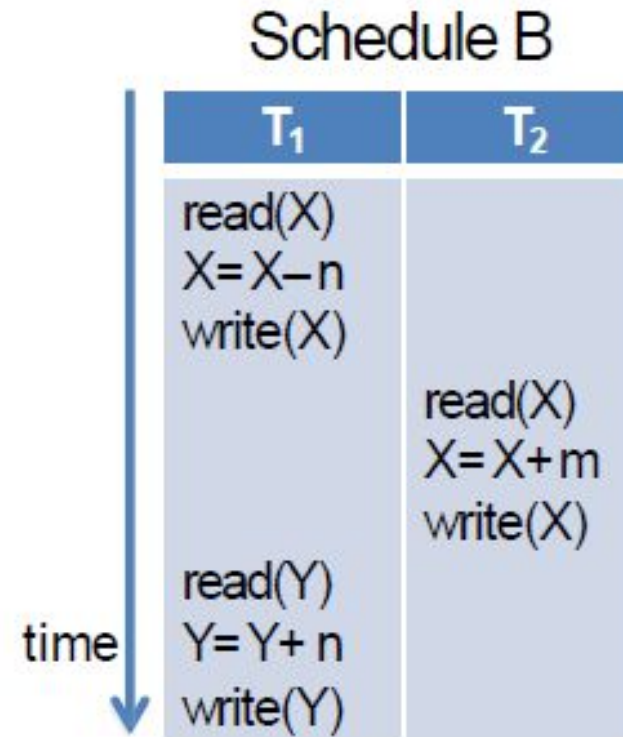
$S_A: r_1(X); w_1(X); r_1(Y); w_1(Y); r_2(X); w_2(X);$

Conflicts:

$r_1(X)$ and $w_2(X)$

$w_1(X)$ and $r_2(X)$

$w_1(X)$ and $w_2(X)$



$S_B: r_1(X); w_1(X); r_2(X); w_2(X); r_1(Y); w_1(Y);$

Conflicts:

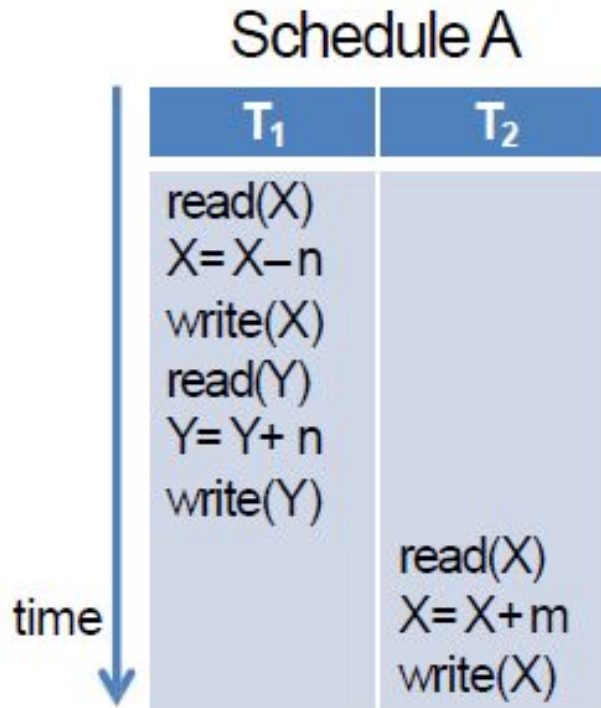
$r_1(X)$ and $w_2(X)$

$w_1(X)$ and $r_2(X)$

$w_1(X)$ and $w_2(X)$



Example 2



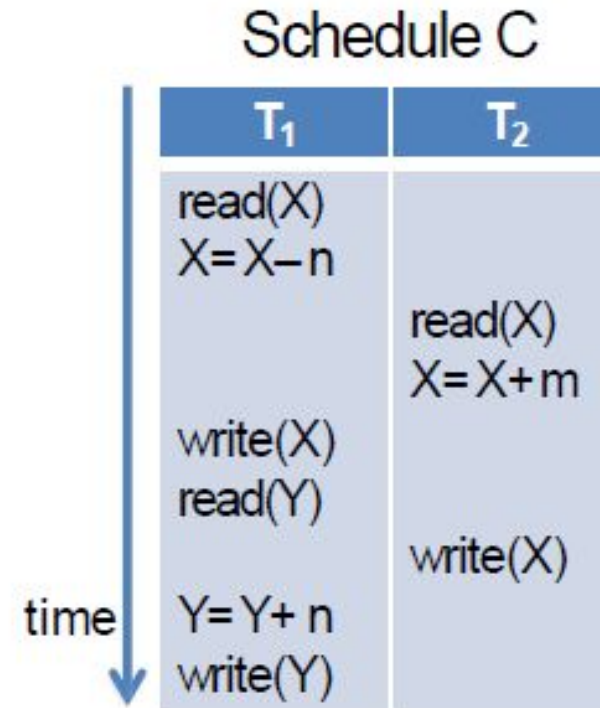
$S_A: r_1(X); w_1(X); r_1(Y); w_1(Y); r_2(X); w_2(X);$

Conflicts:

$r_1(X)$ and $w_2(X)$

$w_1(X)$ and $r_2(X)$

$w_1(X)$ and $w_2(X)$



$S_b: r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); w_1(Y);$

Conflicts:

$r_1(X)$ and $w_2(X)$

$r_2(X)$ and $w_1(X)$

$w_1(X)$ and $w_2(X)$

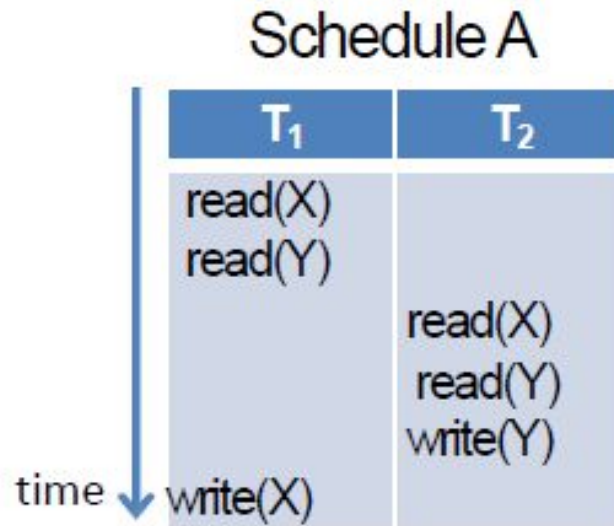


Serializability

- Being able to say that a schedule is serializable, is the same as saying it is correct
 - All serial schedules are correct
 - If schedule S is *equivalent to* a serial schedule
 - Hence, it is also correct
- **Serializable schedules** give the benefit of concurrent execution without giving up correctness
- Determine *serializability* of schedule by swapping non-conflicting operations



Example 3



1) S_A: r₁(X);r₁(Y); r₂(X);r₂(Y);w₂(Y);w₁(X);

2) S_A: r₁(X);r₁(Y); r₂(X);r₂(Y); w₁(X); w₂(Y);

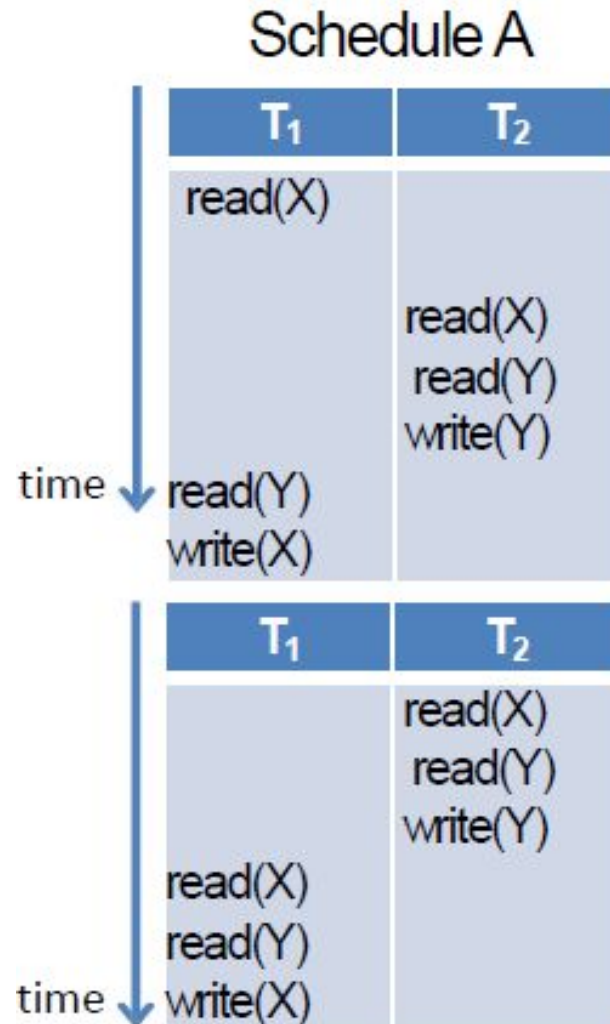
3) S_A: r₁(X);r₁(Y); r₂(X); w₁(X); r₂(Y); w₂(Y);

4) S_A: r₁(X);r₁(Y); r₂(X); w₁(X); r₂(Y); w₂(Y);

- Cannot swap conflicting operations
- Schedule A is not Serializable



Example 4



1. S_A: r₁(X); r₂(X); r₂(Y); w₂(Y) ; r₁(Y); w₁(X);

2. S_A: r₂(X); r₁(X); r₂(Y); w₂(Y) ; r₁(Y); w₁(X);

3. S_A: r₂(X); r₂(Y); r₁(X); w₂(Y) ; r₁(Y); w₁(X);

4. S_A: r₂(X); r₂(Y); w₂(Y); r₁(X); r₁(Y); w₁(X);

Schedule A is Serializable



Serializability

- Most DBMSs will have a set of protocols which:
 - Must be followed by every transaction
 - Are enforced by the concurrency control subsystem
 - Ensure the serializability of all schedules in which the transactions participate
- aka Concurrency Control Protocols



Concurrency Control Protocols



Concurrency Control Protocols

- Locking Protocols
 - Data items are locked to prevent multiple transactions from accessing them concurrently
- Timestamps
 - A unique identifier is generated for each transaction based on the start time
- Optimistic Protocols
 - Based on validation of the transaction after it executes its operations



Locking



Locking

- A lock is a variable associated with a data item
 - Describes the status of the data item with respect to operations that can be applied to it
 - Data items may be at a variety of granularities, e.g. DB, table, tuple, attribute etc.
- Locks are used to synchronise access by concurrent transactions
- There are two main types
 - Binary lock
 - Read/write lock



Binary Lock

- A binary lock can have only two states (or values)
 - _____ and _____
- Two operations are used in a binary lock
 - lock_item
 - unlock_item
- Each transaction locks the item before using it, and then unlocks it when finished



Binary Lock

- A transaction which wants to access a data item requests a lock on that item
 - If the item is unlocked, the transaction locks it and uses it
 - If the item is already locked, the transaction waits until it is unlocked
- Binary locking is rarely used, as it is too restrictive
 - At most, one transaction can access each item
 - Several transactions should be able to access a data item concurrently for read access



Read/Write Lock

- If multiple transactions want to read an item, then they can access the item concurrently
 - Read operations are not conflicting
- However, if a transaction is to write an item, then it must have exclusive access
- The read/write lock implements this form of locking
 - It is called a multiple-mode lock



Read/Write Lock

- There are three locking operations used in read/write locks
 - read_lock
 - write_lock
 - unlock
- A read-locked item is also called a shared lock, as it allows other transactions to also read that item
- A write-locked item is also called an exclusive lock as a single transaction has access to it



Read/Write Lock

- The following rules must be enforced:
 1. A transaction T must issue the operation `read_lock(X)` or `write_lock(X)` before any `read_item(X)` operation is performed in T
 2. A transaction T must issue the operation `write_lock(X)` before any `write_item(X)` operation is performed in T
 3. A transaction T must issue the operation `unlock(X)` after all `read_item(X)` and `write_item(X)` operations are completed in T
 4. A transaction T will not issue an `unlock(X)` operation unless it already holds a read (shared) lock or a write (exclusive) lock on item X



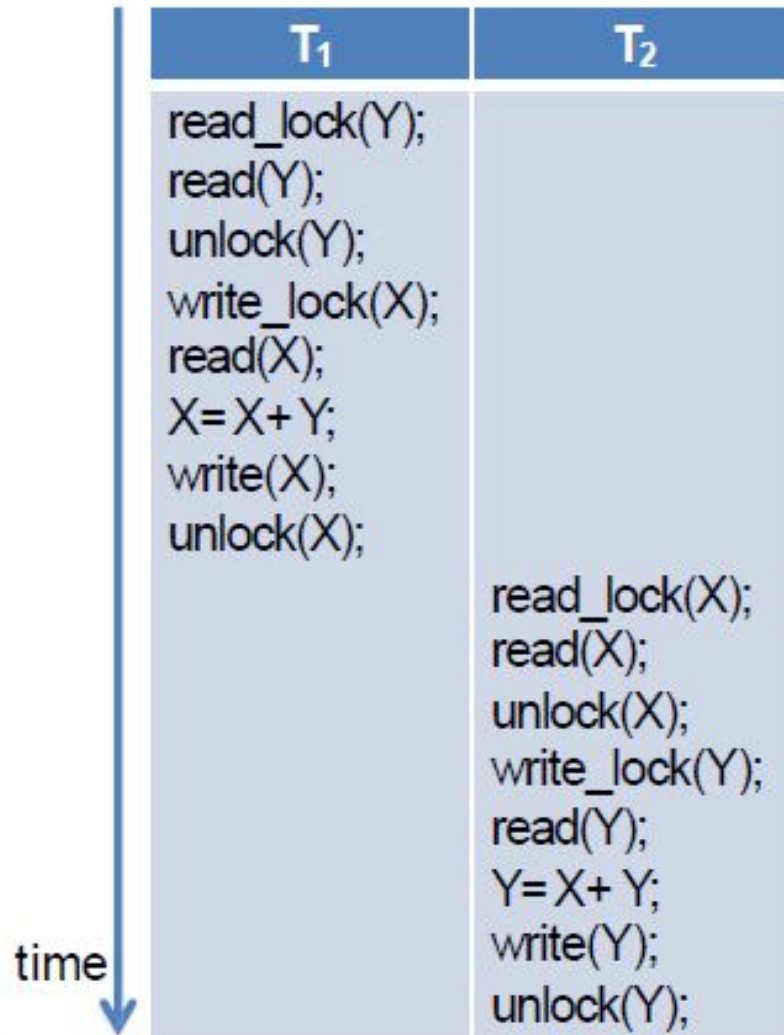
Lock Conversion

- Some DBMS allow the conversion of a lock from one state to another
- A transaction T can issue a `read_lock(X)` and then later *upgrade* the lock by issuing a `write_lock(X)`
 - If T is the only transaction holding a read lock on X at the time it issues the `write_lock(X)` operation, the lock can be upgraded
 - otherwise, the transaction must wait
- It is also possible for a transaction T to issue a `write_lock(X)` and then later to *downgrade* the lock by issuing a `read_lock(X)` operation

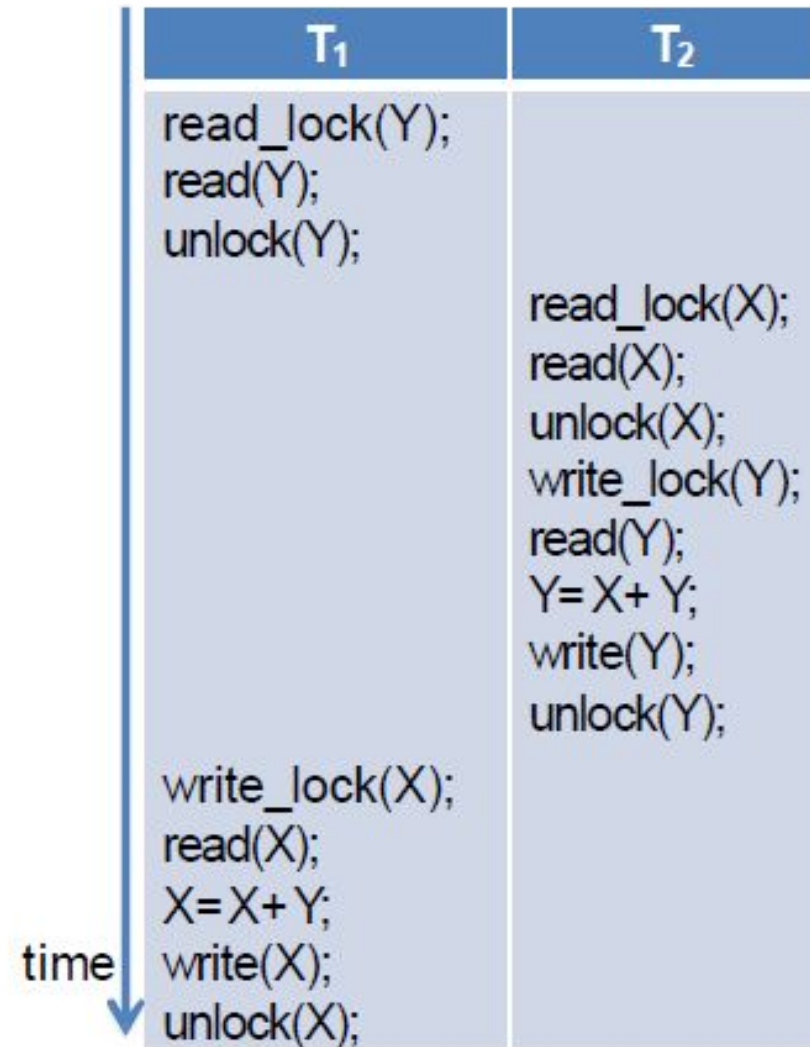


Example

Schedule A



Schedule B



Two-Phase Locking



Two-Phase Locking

- Two-Phase Locking is an additional protocol
 - concerns the positioning of locking and unlocking in every transaction
 - used to guarantee serializability
- A transaction is said to follow two-phase locking if all lock operations (read_lock, write_lock) precede the first unlock operation in that transaction

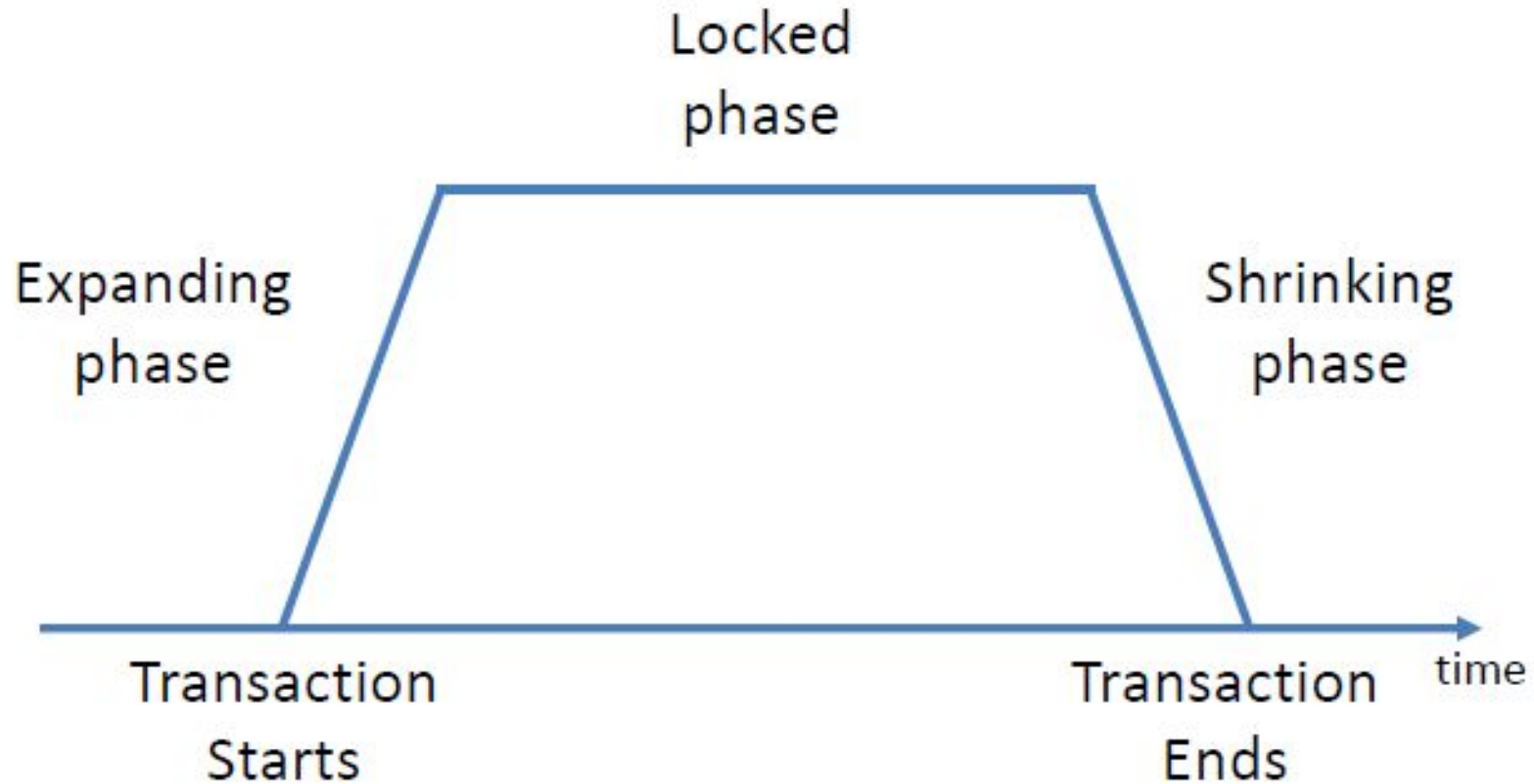


Two-Phase Locking

- Two-Phase Locking transactions are divided into two phases:
 - Expanding phase
 - during which new locks on items can be acquired but none can be released
 - Shrinking phase
 - during which existing locks can be released but no new locks can be acquired.
- If lock conversion is allowed:
 - upgrading of locks (from read-locked to write-locked) must be done during the expanding phase
 - downgrading of locks (from write-locked to read-locked) must be done in the shrinking phase

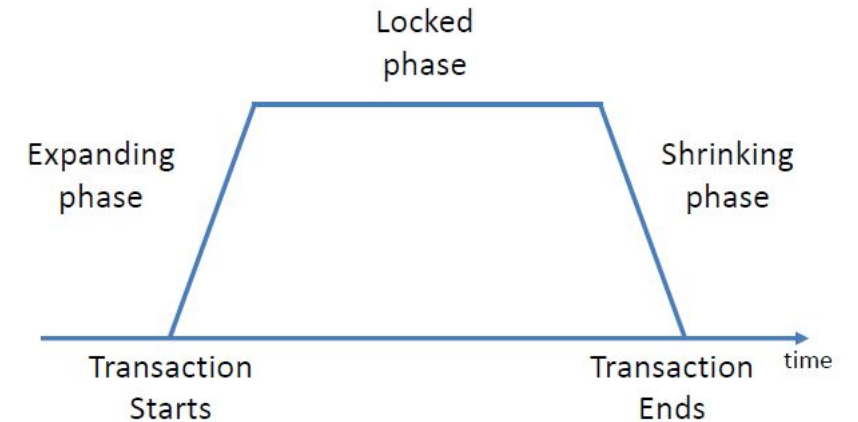
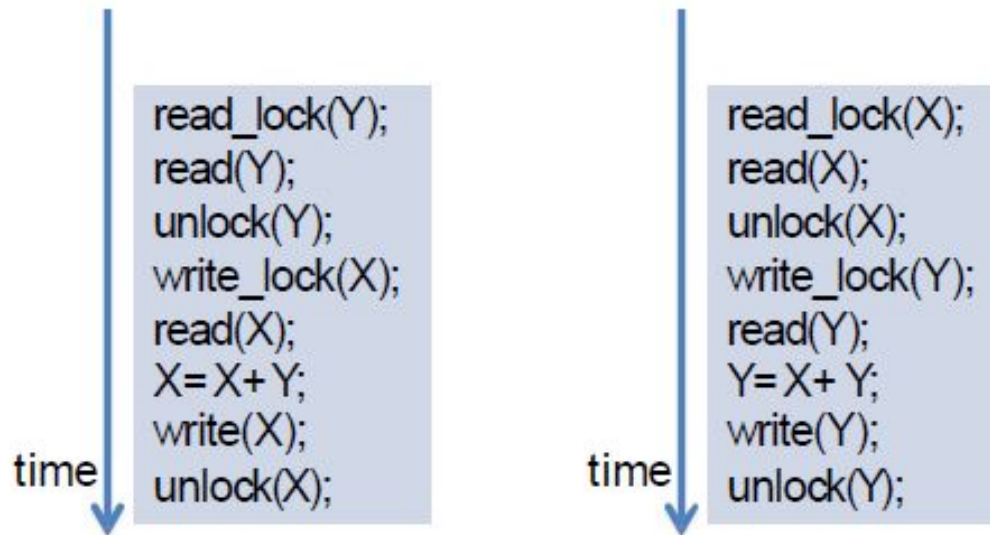


Two-Phase Locking



Two-Phase Locking

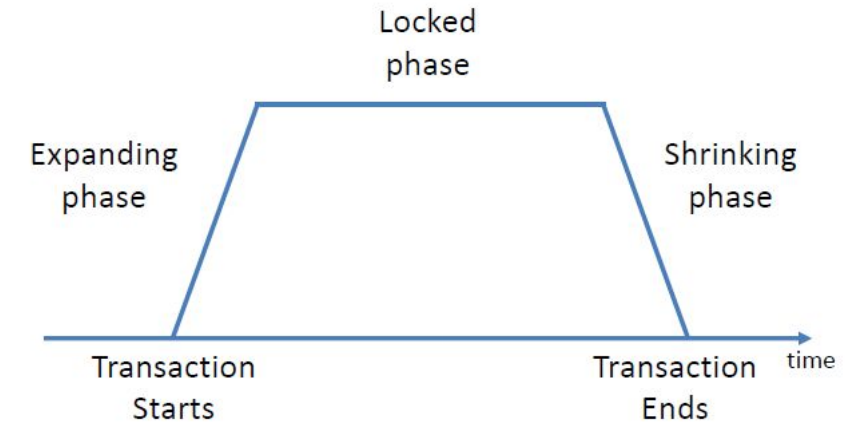
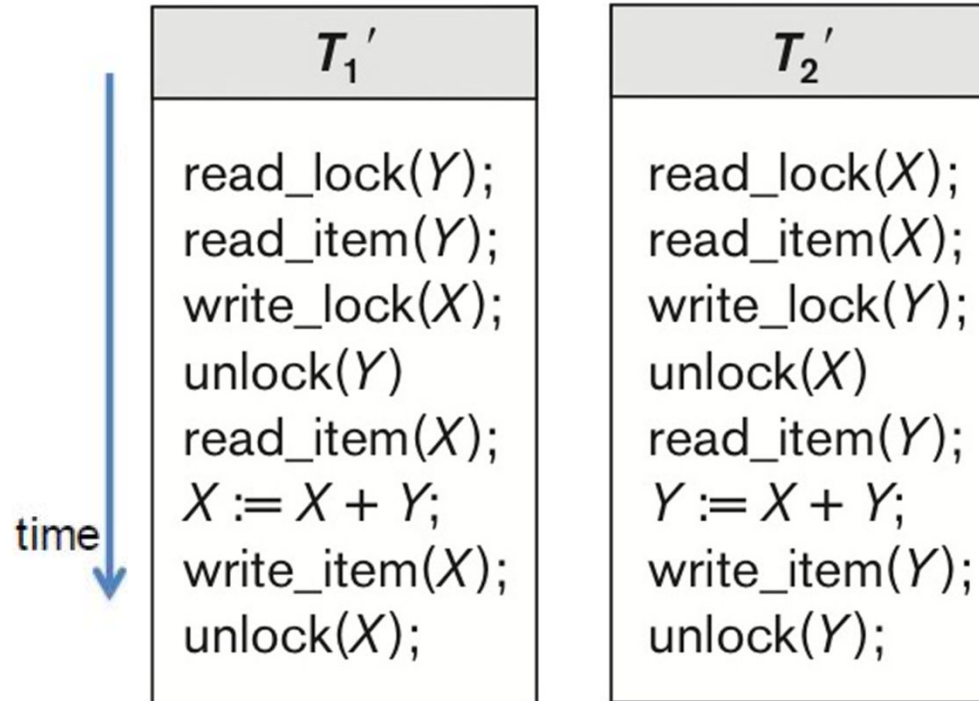
- Consider the transactions from our example:



Do the transactions obey two phase locking?

Two-Phase Locking

- Consider the transactions from our example:



Do the transactions obey two phase locking?

Two-Phase Locking

- It has been proven that:
 - if every transaction in a schedule follows the two-phase locking protocol, the schedule is guaranteed to be *serializable*
- This removes the need to test each schedule produced for serializability
 - However, this comes at a cost



Limitations of Two-Phase Locking

- Can limit the concurrency that can occur in a schedule
- A transaction may not be able to release an item X after it is finished using it, if the transaction must lock an additional item Y at a later point
 - conversely, a transaction may have to lock the additional item Y before it needs it so that it can release X
 - Hence, X must remain locked until all items that the transaction needs to read or write have been locked, only then can X be released
 - Meanwhile, another transaction seeking to access X may be forced to wait



Problems with Locking

- The use of locking in schedules can cause two additional problems:
 - Deadlock
 - Starvation



Deadlock



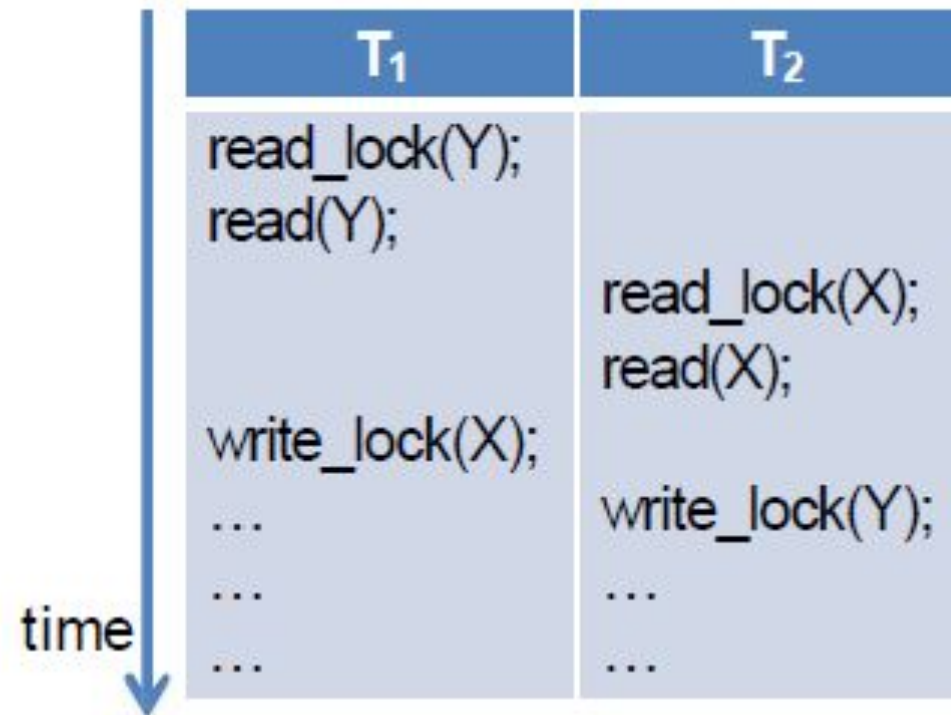
Deadlock

- Deadlock occurs when:
 - each transaction T_i in a set of two or more transactions is waiting for some item that is locked by some other transaction in the set
 - Hence, each transaction in the set is waiting for one of the other transactions in the set to release the lock on an item
 - But because the other transaction is also waiting, it will never release the lock



Deadlock

Schedule A



Deadlock Prevention Protocols

- Used to decide what to do with a transaction involved in a possible deadlock situation:
 - Should it be blocked and made to wait?
 - Should it be aborted?
 - Should the transaction preempt another transaction and cause it to abort?
- No Waiting
- Wait-die
- Wound-wait
- Cautious Waiting



Deadlock Prevention Protocols



Deadlock Prevention Protocols

- Typically use the concept of transaction timestamp
 - unique identifier assigned to each transaction
- Timestamps are based on the order in which transactions are started
 - hence, if transaction T1 starts before transaction T2, then $TS(T1) < TS(T2)$.



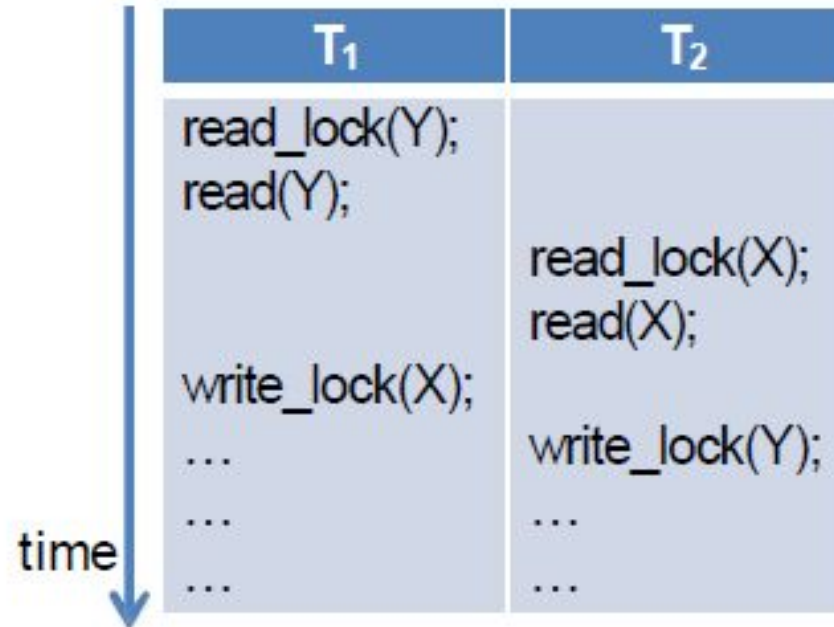
No Waiting

- In the *no waiting* approach, if a transaction is unable to obtain a lock, it is immediately aborted
- It is then restarted after a certain time delay without checking whether a deadlock will actually occur or not
- In this case, no transaction ever waits, so no deadlock will occur
 - However, this scheme can cause transactions to abort and restart needlessly



No Waiting

Schedule A



- Transaction T₁ will abort as it is unable to obtain the lock on X, that way T₂ never encounters a block on locking Y

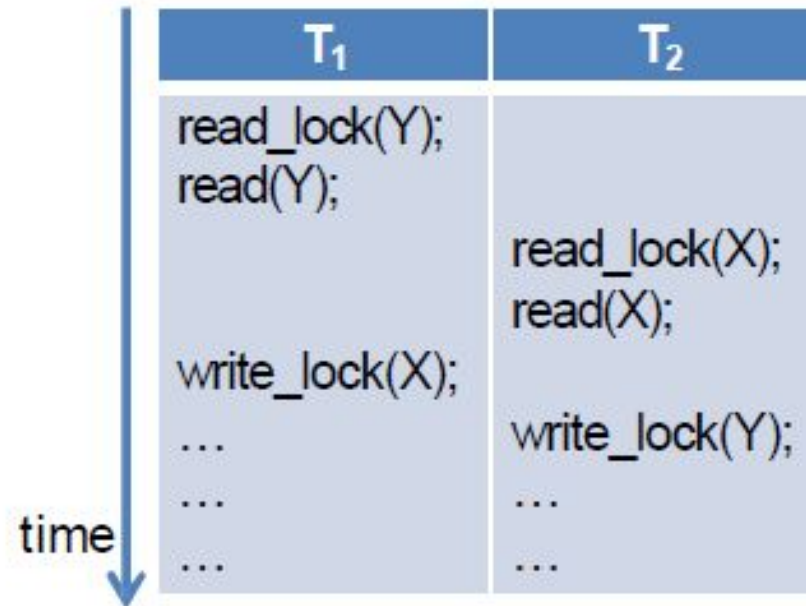
Wait-die

- Suppose that transaction T_a tries to lock an item X but is not able to because X is locked by some other transaction T_b
- The rules followed by *Wait-die* are:
 - If T_a is older than T_b then T_a is allowed to wait
 - otherwise, if T_a is younger than T_b then abort T_a and restart it later with the same timestamp
 - So T_a either waits or dies



Wait die

Schedule A



- Transaction T_1 is older than T_2 and thus is allowed to wait after it is unable to obtain the lock on X
- Transaction T_2 is younger than T_1 and thus is aborted after it is unable to obtain the lock on Y

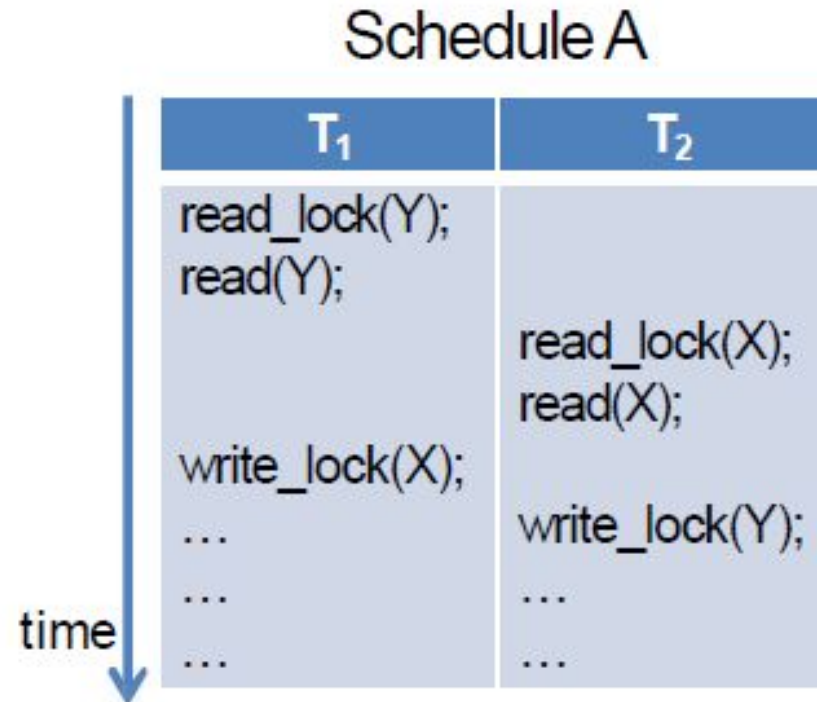


Wound-wait

- Suppose that transaction T_a tries to lock an item X but is not able to because X is locked by some other transaction T_b
- The rules followed by *Wound-wait* are:
 - If T_a is older than T_b then abort T_b and restart it later with the same timestamp
 - otherwise, if T_a is younger than T_b then T_a is allowed to wait
 - So T_a either wounds T_b or waits



Wound-wait



- T_1 is older than T_2 and thus T_2 is aborted after T_1 is unable to obtain the lock on X
 - T_2 is “wounded”

Wait-die and Wound-wait

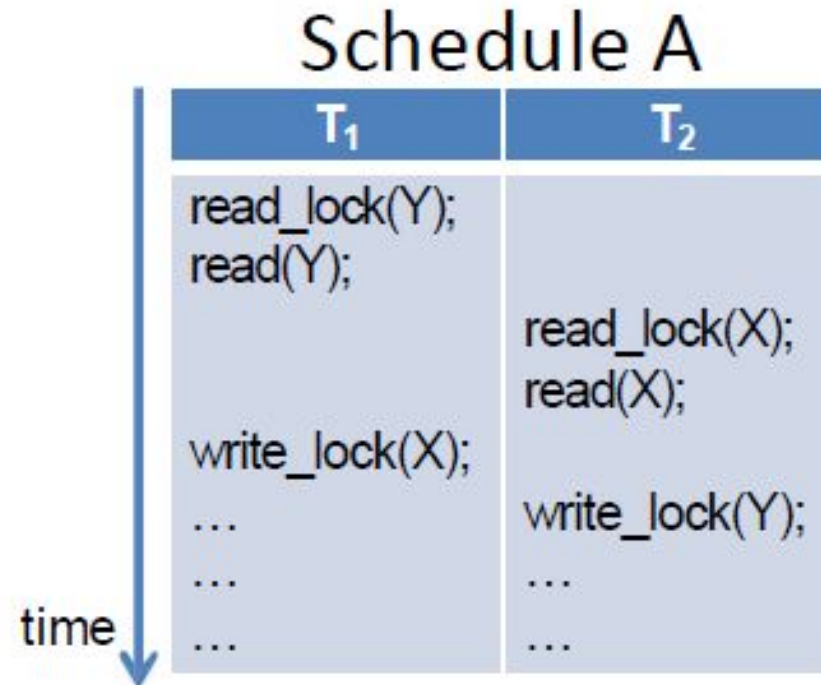
- Of the two transactions that may be involved in a deadlock, both these approaches abort the transaction that started later
 - assumption is that this will waste less processing
- However, both may cause some transactions to be aborted and restarted needlessly
 - even though those transactions may never actually cause a deadlock.



Cautious Waiting

- Proposed to try to reduce the number of needless aborts
- Suppose that transaction T_a tries to lock an item X but is not able to do so because X is locked by some other transaction T_b
- The cautious waiting rules are:
 - If T_b is not blocked (not waiting for some other locked item), then T_a is blocked and allowed to wait
 - otherwise abort T_a

Cautious Waiting



- When T_1 is unable to obtain the lock on X , T_2 which holds the lock on X is not blocked, so T_1 can wait
- When T_2 is unable to obtain the lock on Y , T_1 which holds the lock on Y is also blocked, so T_2 is aborted

Deadlock Detection



Deadlock Detection

- Deadlock detection is used to check if a state of deadlock actually exists
 - attractive if there is little interference among transactions i.e. if different transactions rarely access the same items at the same time
 - however, if transactions are long and each transaction uses many items, it may be better to use a deadlock prevention protocol



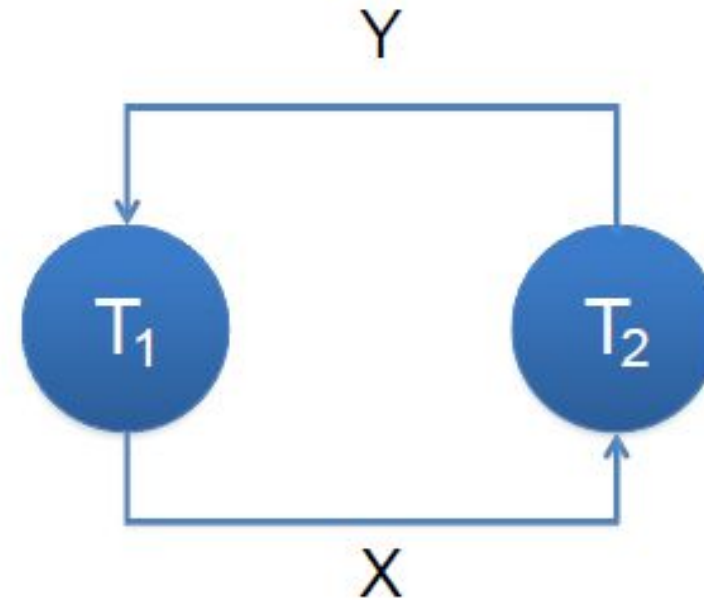
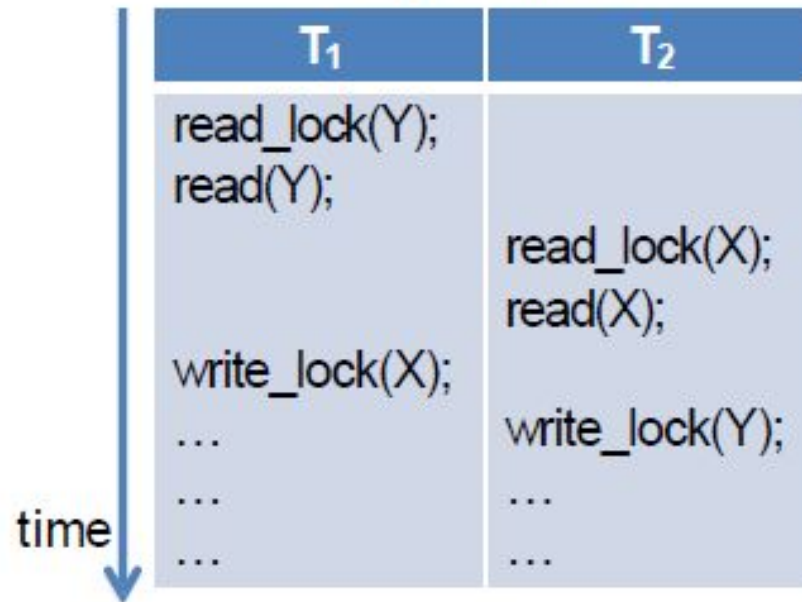
Deadlock Detection

- A simple way to detect deadlock is for the system to construct and maintain a *wait-for graph*
 - One node is created in the wait-for graph for each transaction that is currently executing
 - Whenever a transaction T_a is waiting to lock an item X that is currently locked by a transaction T_b , a directed edge ($T_a \rightarrow T_b$) is created in the wait-for graph
 - When T_b releases the lock(s) on the items that T_a is waiting for, the directed edge is dropped from the wait-for graph.
- There is a state of deadlock if, and only if, the wait-for graph has a loop



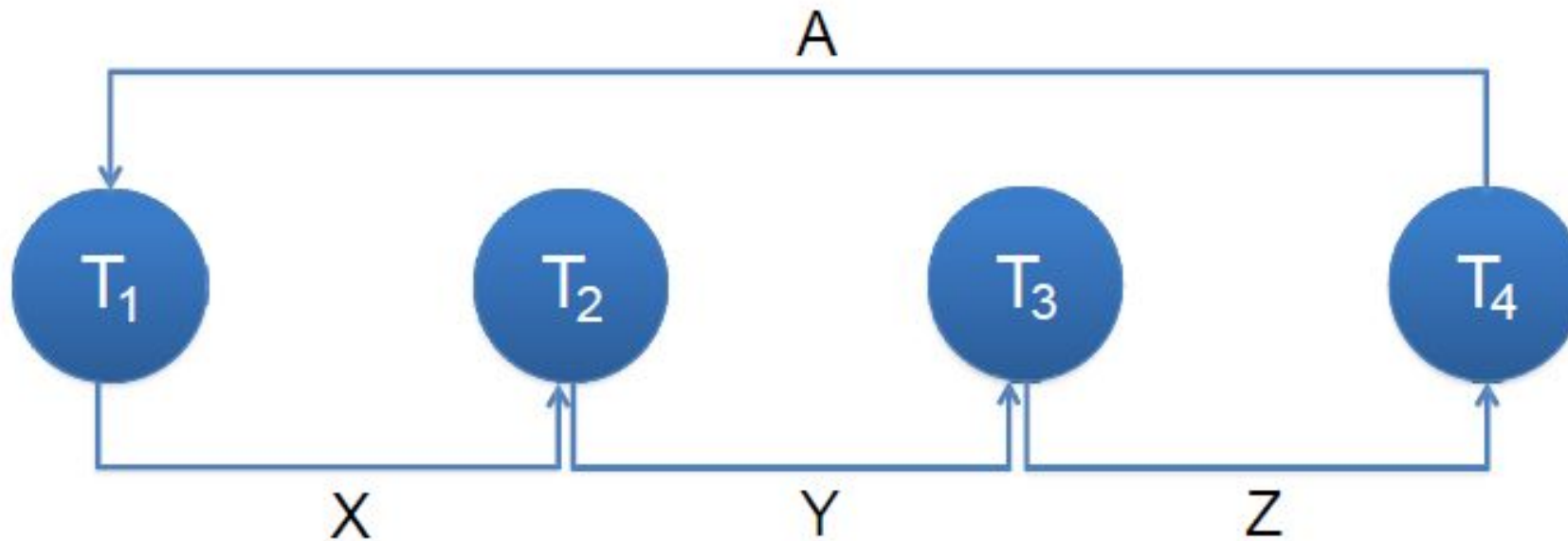
Deadlock Detection

Schedule A



Deadlock Detection

- Deadlock can occur between two transactions indirectly via a chain of intermediate transactions



Deadlock Detection

- Once detected, deadlock must be resolved by aborting and rolling back one of the transactions
- Choosing which transactions to abort is known as *victim selection*
 - avoid selecting transactions that have run for a long time or have performed many updates
 - instead, select transactions that have not made many changes (younger transactions)



Starvation



Starvation

- Another problem that may occur when using locking is *starvation*
 - when a transaction cannot proceed for an indefinite period of time while other transactions in the system continue normally
- Can occur if the waiting scheme for locked items gives priority to some transactions over others
- Can also occur because of victim selection
 - if the algorithm repeatedly selects the same victim transaction, causing it to abort and never finish



Starvation Solutions

- first-come-first-served queue
 - transactions are allowed to lock an item in the order in which they originally requested the lock
- waiting list
 - allows some transactions to have priority over others but increases the priority of a transaction the longer it waits, until it eventually gets the highest priority and proceeds
- victim priority
 - assigns higher priorities to transactions that have been aborted multiple times



Timestamp Ordering



Timestamp Ordering

- As already discussed, timestamp values are assigned to transactions in the order they are submitted to the system
 - so a timestamp can be thought of as the transaction start time
- There are concurrency control techniques based purely on timestamp ordering which do not use locking
 - hence, deadlocks cannot occur



Timestamp Ordering

- For each data item accessed by conflicting operations in the schedule, the order in which the item is accessed must not violate the timestamp ordering
- This produces serializable schedules which are equivalent to the serial schedule
 - as all conflicting operations are conducted in the same order that they would be in the serial schedule



Timestamp Ordering

- In order to enforce this, the DBMS keeps two timestamp values for each data item:
 - $\text{read_TS}(X)$ – This is equal to the timestamp of the most recently started (youngest) transaction that has successfully read item X
 - $\text{write_TS}(X)$ – This is equal to the timestamp of the most recently started (youngest) transaction that has successfully written item X

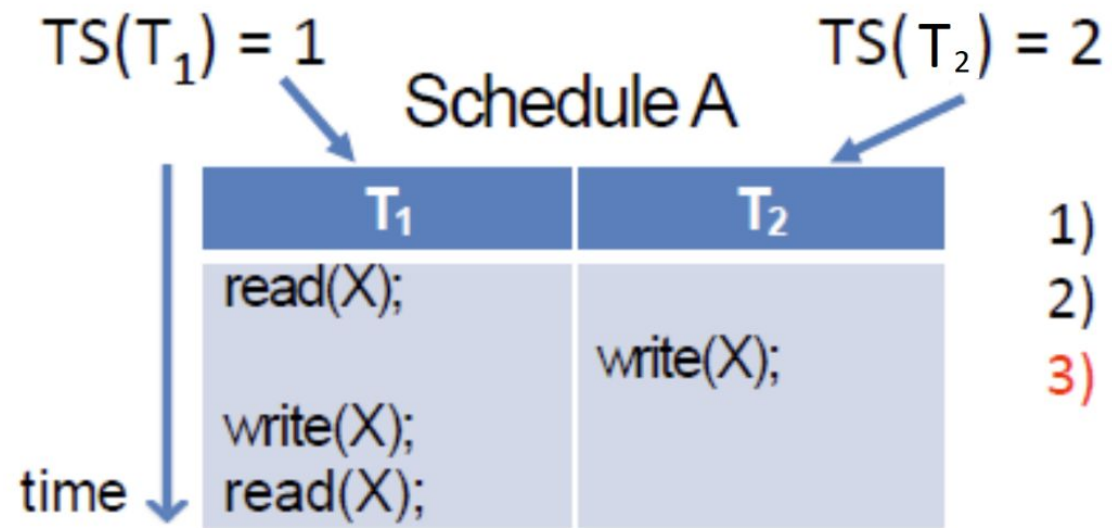


Timestamp Ordering

- Whenever a transaction T issues a $\text{write_item}(X)$ operation, the following is checked:
 - If $\text{read_TS}(X) > \text{TS}(T)$ or if $\text{write_TS}(X) > \text{TS}(T)$, then abort and rollback T and reject the operation
 - This should be done because some younger transaction with a timestamp greater than $\text{TS}(T)$ – and hence after T in the timestamp ordering – has already read or written the value of item X before T had a chance to write X , thus violating the timestamp ordering
 - If $\text{read_TS}(X) \leq \text{TS}(T)$ and if $\text{write_TS}(X) \leq \text{TS}(T)$, then execute the $\text{write_item}(X)$ operation of T and set $\text{write_TS}(X)$ to $\text{TS}(T)$



Timestamp Ordering



- 1) read_TS(X) = 1
- 2) write_TS(X) = 2
- 3) write_TS(X)
 - Violation: write_TS(X) > TS(T_1)
 - T_1 cannot overwrite update by T_2
 - Rollback T_1 and reject operation



Timestamp Ordering

- Whenever a transaction T issues a $\text{read_item}(X)$ operation, the following is checked:
 - If $\text{write_TS}(X) > \text{TS}(T)$, then abort and rollback T and reject the operation
 - This should be done because some younger transaction with timestamp greater than $\text{TS}(T)$ – and hence after T in the timestamp ordering – has already written the value of item X before T had a chance to read X
 - If $\text{write_TS}(X) \leq \text{TS}(T)$, then execute the $\text{read_item}(X)$ operation of T and set $\text{read_TS}(X)$ equal to the larger of $\text{TS}(T)$ and the current $\text{read_TS}(X)$



Summary

Transaction

- Properties / Failure

Concurrency Control

- Lost Updates
- Temporary Updates
- Incorrect Summary

Schedules

- Schedule Conflicts
- Serial Schedules
- Issues with Serial Schedules

Serializability

Result Equivalence

Conflict Equivalence

Locking

Binary Lock

Read/Write Lock

Lock Conversion

Two Phase Locking

- Problems/Limitations

Deadlock

Deadlock Prevention

- No Waiting
- Wait-die
- Wound-wait
- Cautious Waiting

Deadlock Detection

Starvation

Starvation Solutions

Timestamp Ordering



CSU34041

Transactions and Concurrency Control

Yvette Graham
ygraham@tcd.ie



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin